

YouTube

Device Authorization

Using a QR Code

API Doc

OAuth 2.0 Device Flow • QR Handoff • YouTube Data API

The QR code simplifies the handoff to a phone. Google OAuth performs the actual user authorization.

1. Overview

This document describes an authentication architecture that allows a user to authorize a television, game console, streaming device, or other limited-input device to access the YouTube Data API by using a smartphone and a QR code.

The implementation combines:

- YouTube Data API
- Google OAuth 2.0 Device Authorization Flow
- QR-code generation
- A smartphone or other browser-enabled secondary device
- Token polling and secure token storage

The QR code acts as a convenient bridge between the limited-input device and the browser-based Google authorization process. The QR code itself does not authenticate the device; authentication and authorization are performed by Google's OAuth 2.0 service.

2. Use Case

Consider a smart-TV application that needs permission to access a user's YouTube account. Entering an email address, password, and authentication information with a television remote would create a poor user experience and would require the application to handle credentials that it should never receive.

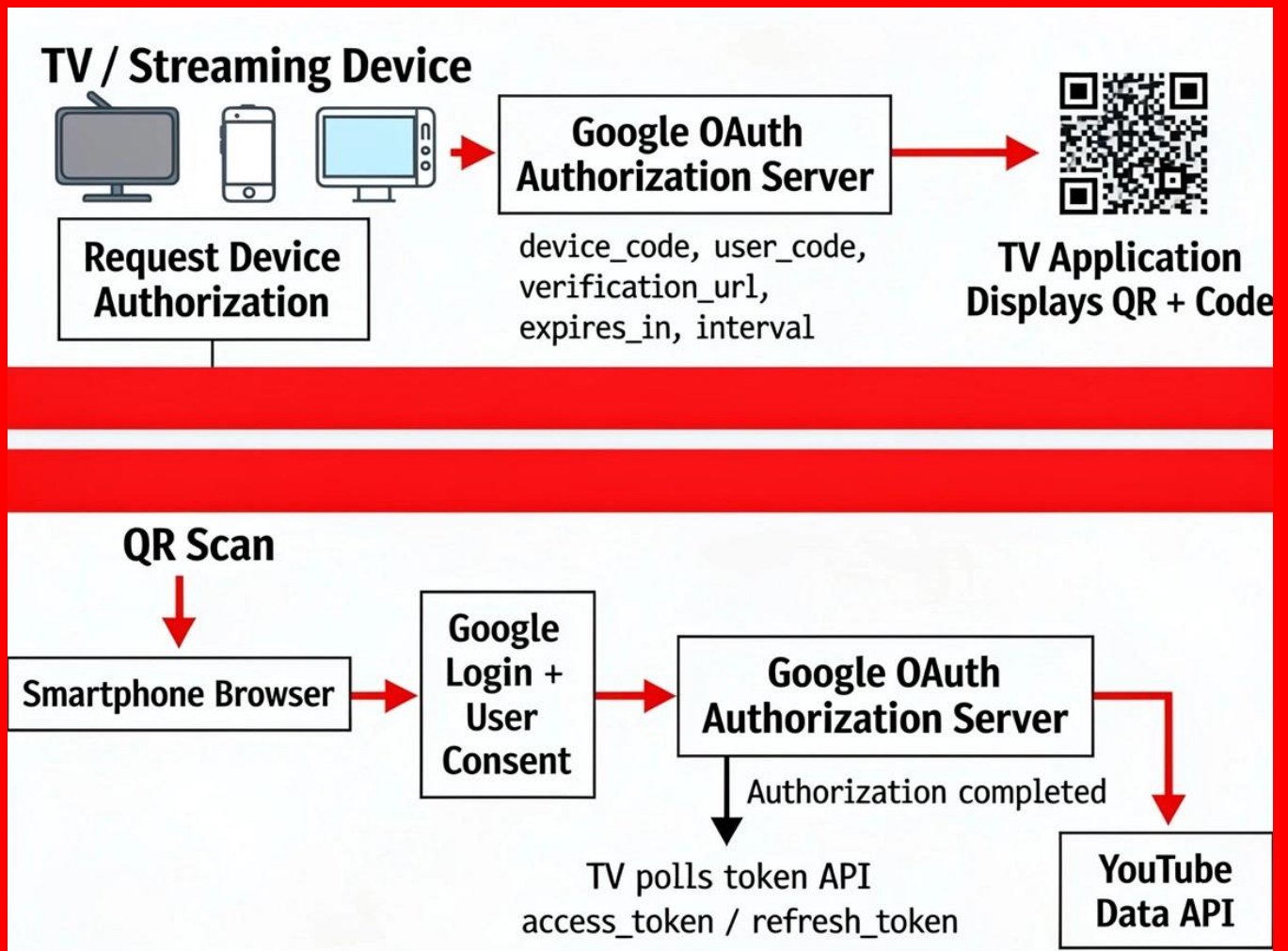
Instead, the application displays:

CONNECT YOUR YOUTUBE ACCOUNT

1. Scan the QR code with your phone.
2. Follow the Google sign-in instructions.
3. Enter the code shown on your TV if requested.
4. Approve access.
5. Return to the TV.

The television automatically detects successful authorization and continues into the application.

3. Architecture



4. OAuth Device Authorization Flow

Google provides an OAuth 2.0 authorization flow for devices with limited input capabilities, including televisions, game consoles, cameras, and similar devices. The application first requests authorization codes from Google's device authorization endpoint.

Endpoint

POST <https://oauth2.googleapis.com/device/code>

Example Request

```
POST /device/code HTTP/1.1
Host: oauth2.googleapis.com
Content-Type: application/x-www-form-urlencoded

client_id=YOUR_CLIENT_ID&
scope=https://www.googleapis.com/auth/youtube.readonly
```

For a read-only YouTube implementation, the application can request:

```
https://www.googleapis.com/auth/youtube.readonly
```

5. Device Authorization Response

Google returns information similar to:

```
{
  "device_code": "DEVICE_CODE_VALUE",
  "user_code": "ABCD-EFGH",
  "verification_url": "https://www.google.com/device",
  "expires_in": 1800,
  "interval": 5
}
```

Field	Purpose
device_code	Identifies the authorization session to the OAuth server.
user_code	Short code presented to the user.
verification_url	Google URL where authorization takes place.
expires_in	Number of seconds before the authorization request expires.
interval	Minimum interval between token polling requests.

The device_code should never be treated as the code intended for manual user entry. The user interacts with the shorter user_code.

6. QR-Code Integration

After receiving the OAuth response, the device generates a QR code. The simplest architecture encodes the returned verification URL:

`https://www.google.com/device`

The device simultaneously displays the associated user_code.

CONNECT TO **YouTube**

[QR CODE]

Scan with your phone

Verification code:
ABCD-EFGH

Code expires in 30 minutes.

In production, the application should use the verification URL returned by Google rather than hard-coding or modifying it.

7. Smartphone Authorization

The smartphone performs the user-facing portion of authentication. The user:

- Scans the QR code.
- Opens Google's device-verification page.
- Signs in to their Google account if necessary.
- Enters the user_code displayed on the television.
- Reviews the application's requested permissions.
- Selects Allow or Deny.

At no point does the television application receive the user's Google password.

8. Device Polling

Polling means your program repeatedly sends API requests to check for new information.

While the user completes authorization on the phone, the original device polls Google's OAuth token endpoint.

Endpoint

POST <https://oauth2.googleapis.com/token>

Conceptually, the request contains:

```
client_id
device_code
grant_type
```

with the device authorization grant type:

```
urn:ietf:params:oauth:grant-
type:device_code
```

The device should wait for the polling interval provided in the initial authorization response rather than repeatedly sending requests as quickly as possible.

interval = 5 seconds

0 sec → Poll

5 sec → Poll

10 sec → Poll

15 sec → Poll

20 sec → Authorization detected

9. Polling Logic

Example pseudocode:

```
while not authorization_complete:

    response = request_access_token(
        client_id=CLIENT_ID,
        device_code=device_code
    )

    if response.status == "authorization_pending":
        sleep(polling_interval)

    elif response.status == "slow_down":
        increase_polling_interval()

    elif response.status == "access_denied":
        display_authorization_denied()
        break

    elif response.status == "expired_token":
        restart_device_authorization()
        break

    elif response.access_token:
        save_tokens_securely(response)
        authorization_complete = True
```

10. Successful Authorization

After the user approves the request, Google's token endpoint returns an OAuth access token. A successful response conceptually resembles:

```
{
  "access_token": "ACCESS_TOKEN",
  "expires_in": 3600,
  "refresh_token": "REFRESH_TOKEN",
  "scope": "https://www.googleapis.com/auth/youtube.readonly",
  "token_type": "Bearer"
}
```

11. Calling the YouTube Data API

For example, the application can retrieve information about the authenticated user's channel:

```
GET /youtube/v3/channels?part=snippet&mine=true HTTP/1.1
Host: www.googleapis.com
Authorization: Bearer ACCESS_TOKEN
```

The authorization token should preferably be sent in the HTTP Authorization header rather than in the URL.

12. QR-Code Generation Example

A backend or device application could generate the QR code programmatically.

```
import qrcode

verification_url = "https://www.google.com/device"

qr = qrcode.QRCode(
    version=1,
    box_size=10,
    border=4
)

qr.add_data(verification_url)
qr.make(fit=True)

image = qr.make_image()
image.save("youtube_device_verification.png")
```

In production, `verification_url` should come from Google's authorization response rather than being assumed.

13. END TO END APPLICATION LOGIC

```
def connect_youtube_device():

    auth = request_device_code()

    device_code = auth["device_code"]
    user_code = auth["user_code"]
    verification_url = auth["verification_url"]
    interval = auth["interval"]

    qr_code = generate_qr(verification_url)

    display_connection_screen(
        qr_code=qr_code,
        user_code=user_code
    )

    while True:
        result = poll_google(device_code=device_code)

        if result == "authorization_pending":
            sleep(interval)
            continue

        if result == "slow_down":
            interval += 5
            sleep(interval)
            continue

        if result == "access_denied":
            show_error("Authorization denied")
            return

        if result == "expired_token":
            restart_connection()
            return

        if result.get("access_token"):
            securely_store_tokens(result)
            break

    load_youtube_account()
```

14. Security Requirements

Never place an access token inside the QR code

The QR code should facilitate the authorization journey. It should not contain:

- access_token
- refresh_token
- Google password
- API secret
- private session credentials

Use HTTPS

All browser-facing endpoints should use HTTPS rather than unencrypted HTTP.

Protect refresh tokens

Refresh tokens may permit long-lived access to the user's account. They should be:

- encrypted at rest
- restricted to the authorized device or user session where practical
- excluded from application logs
- excluded from analytics events
- removed when account access is disconnected

Limit OAuth Scopes

Applications should request only the permissions required for their functionality. For read-only YouTube access:

```
https://www.googleapis.com/auth/youtube.readonly
```

15. ERROR HANDLING

Authorization Pending

Waiting for authorization... Scan the QR code and complete sign-in on your phone.

User Denied Access

YouTube connection was not approved. Try Again.

Code Expired

This connection code has expired. Generate New Code.

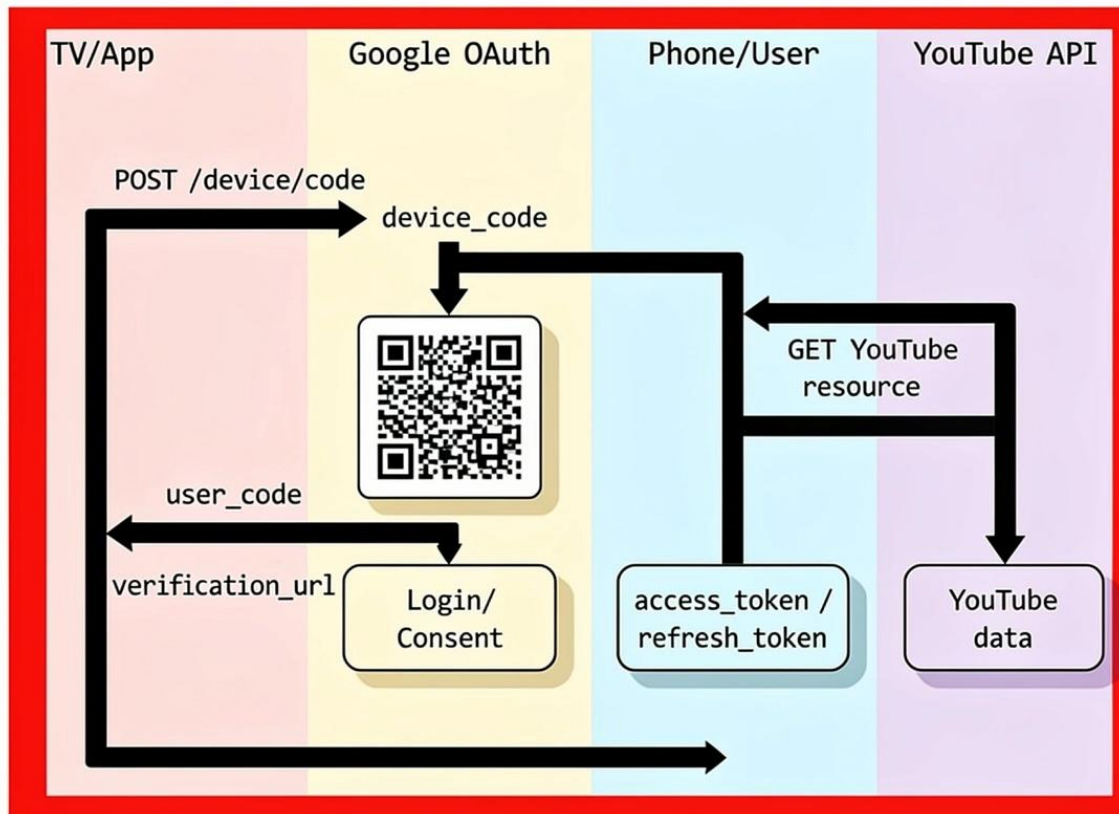
Network Failure

Unable to contact the authorization service. Check your internet connection and try again.

API Authentication Failure

If an access token has expired, the application should attempt the approved token-refresh process before asking the user to reconnect.

16. Sequence Diagram



17. Important Design Distinction

QR CODE



Helps the user reach the authorization interface

USER CODE



Associates the user's browser authorization with the device authorization session

DEVICE CODE



Allows the application to poll Google's OAuth server for authorization status

ACCESS TOKEN



Allows authenticated calls to the YouTube Data API

This separation prevents the device from collecting the user's Google credentials.

18. Recommended UX

Connect YouTube

Use your phone to scan this code.



After successful OAuth authorization:

✓ YouTube connected

Your device has been successfully authorized.

Continue

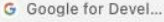
19. zAuth account Python code

If you're adding an account, there is cloud data shared between the YouTube users to be aware of.

QR-Code Device Authorization with OAuth Polling

Yes — for a **YouTube TV → phone authorization flow**, the TV can receive **JSON** from Google's OAuth device authorization endpoint, and Swift can decode that JSON into Swift structs.

Google's documented flow is:

TV app → request device code → Google returns JSON → TV shows QR/code → phone authorizes → TV polls Google → Google returns access token.  +1

Here's what the JSON-to-Swift part looks like.

```
</> JSON    
  
{  
  "device_code": "AH-1Ng...",  
  "user_code": "ABCD-EFGH",  
  "verification_url": "https://www.google.com/device",  
  "expires_in": 1800,  
  "interval": 5  
}
```

In Swift:

```
</> Swift    
  
struct DeviceAuthorizationResponse: Codable {  
    let deviceCode: String  
    let userCode: String  
    let verificationURL: String  
    let expiresIn: Int  
    let interval: Int  
  
    enum CodingKeys: String, CodingKey {  
        case deviceCode = "device_code"  
        case userCode = "user_code"  
        case verificationURL = "verification_url"  
        case expiresIn = "expires_in"  
        case interval  
    }  
}
```

Then decode Google's JSON:

```
</> Swift
let decoder = JSONDecoder()

do {
    let authResponse = try decoder.decode(
        DeviceAuthorizationResponse.self,
        from: jsonData
    )

    print(authResponse.deviceCode)
    print(authResponse.userCode)
    print(authResponse.verificationURL)

} catch {
    print("JSON decoding error: \(error)")
}
```

The TV initially makes a `POST` request to Google's device authorization endpoint:

```
</> Swift
let url = URL(
    string: "https://oauth2.googleapis.com/device/code"
)!

var request = URLRequest(url: url)
request.httpMethod = "POST"

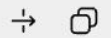
let body =
    "client_id=YOUR_CLIENT_ID" +
    "&scope=https://www.googleapis.com/auth/youtube.readonly"

request.httpBody = body.data(using: .utf8)

request.setValue(
    "application/x-www-form-urlencoded",
    forHTTPHeaderField: "Content-Type"
)
```

Then:

</> Swift



```
URLSession.shared.dataTask(with: request) {  
    data,  
    response,  
    error in  
  
    guard let data = data else {  
        return  
    }  
  
    do {  
        let result = try JSONDecoder().decode(  
            DeviceAuthorizationResponse.self,  
            from: data  
        )  
  
        print("User code:", result.userCode)  
        print("Verification URL:",  
            result.verificationURL)  
    } catch {  
        print(error)  
    }  
}
```

20. Summary

The YouTube QR-device authorization architecture separates account authentication from the limited-input device.

1. Requests a device authorization session from Google.
2. Receives a device_code, user_code, and verification_url.
3. Converts the verification URL into a QR code.
4. Displays the QR code and user code.
5. Polls Google's OAuth token endpoint.
6. Detects authorization after the user approves access from their phone.
7. Receives an OAuth access token.
8. Uses the access token to communicate with the YouTube Data API.
9. Uses the refresh-token process when continued authorization is required.

This design provides a secure and user-friendly method for connecting a YouTube account to televisions, game consoles, streaming devices, and other applications where conventional browser-based login is inconvenient.

References

Google Developers - OAuth 2.0 for TV and Limited-Input Device Applications

<https://developers.google.com/youtube/v3/guides/auth/devices>

Google OAuth 2.0 Device Authorization endpoint: <https://oauth2.googleapis.com/device/code>